

rd57gialv

November 18, 2025

```
[37]: # 1. Load the sample dataset provided in text_class.csv.
import pandas as pd
df = pd.read_excel(r'C:\Users\LENOVO\Desktop\NLP\NLP Final project\part B- News_
↳Article Classification\data_news.xlsx')
print(df)
```

	category	headline \
0	WELLNESS	143 Miles in 35 Days: Lessons Learned
1	WELLNESS	Talking to Yourself: Crazy or Crazy Helpful?
2	WELLNESS	Crenezumab: Trial Will Gauge Whether Alzheimer...
3	WELLNESS	Oh, What a Difference She Made
4	WELLNESS	Green Superfoods
...	...	...
49995	SPORTS	This Baseball Team Learned There's A Wrong Way...
49996	SPORTS	Some Young Spurs Fan Dabbed 38 Times In A Sing...
49997	SPORTS	Rasheed Wallace Ejected From Knicks-Suns Game ...
49998	SPORTS	Why Jake Plummer And Other NFL Players Are Pus...
49999	SPORTS	Simone Biles Isn't The Next Anyone, She's 'The...

	links \
0	https://www.huffingtonpost.com/entry/running-l...
1	https://www.huffingtonpost.com/entry/talking-t...
2	https://www.huffingtonpost.com/entry/crenezuma...
3	https://www.huffingtonpost.com/entry/meaningfu...
4	https://www.huffingtonpost.com/entry/green-sup...
...	...
49995	https://www.huffingtonpost.com/entry/san-jose-...
49996	https://www.huffingtonpost.com/entry/dab-kid-s...
49997	https://www.huffingtonpost.com/entry/rasheed-w...
49998	https://www.huffingtonpost.comhttp://extras.de...
49999	https://www.huffingtonpost.com/entry/the-first...

	short_description \
0	Resting is part of training. I've confirmed wh...
1	Think of talking to yourself as a tool to coac...
2	The clock is ticking for the United States to ...
3	If you want to be busy, keep trying to be perf...
4	First, the bad news: Soda bread, corned beef a...

```

...
49995 Many fans were pissed after seeing the minor l...
49996         Never change, young man. Never change.
49997 Wallace was hit with a first technical for a h...
49998 They believe CBD could be an alternative to po...
49999         The gymnast is in a league of her own.

```

```

                                keywords
0                                running-lessons
1                                talking-to-yourself-crazy
2                                crenezumab-alzheimers-disease-drug
3                                meaningful-life
4                                green-superfoods
...
49995        san-jose-giants-japanese-heritage-night
49996                                dab-kid-san-antonio-spurs
49997 rasheed-wallace-ejected-knicks-suns-ball-dont-lie
49998                                NaN
49999        the-first-simone-biles

```

[50000 rows x 5 columns]

```

[38]: # dropping duplicate if any
df = df.drop_duplicates()
df = df.reset_index(drop=True)
display(df)

```

```

category                                headline \
0  WELLNESS        143 Miles in 35 Days: Lessons Learned
1  WELLNESS        Talking to Yourself: Crazy or Crazy Helpful?
2  WELLNESS  Crenezumab: Trial Will Gauge Whether Alzheimer...
3  WELLNESS                                Oh, What a Difference She Made
4  WELLNESS                                Green Superfoods
...
45744  SPORTS  Gregg Popovich Wishes He 'Had The Guts' To Dye...
45745  SPORTS        Rafal Lipinski Nails Sick, Complicated Dunk
45746  SPORTS  This Baseball Team Learned There's A Wrong Way...
45747  SPORTS  Some Young Spurs Fan Dabbed 38 Times In A Sing...
45748  SPORTS  Simone Biles Isn't The Next Anyone, She's 'The...

links \
0  https://www.huffingtonpost.com/entry/running-l...
1  https://www.huffingtonpost.com/entry/talking-t...
2  https://www.huffingtonpost.com/entry/crenezuma...
3  https://www.huffingtonpost.com/entry/meaningfu...
4  https://www.huffingtonpost.com/entry/green-sup...
...
45744  https://www.huffingtonpost.com/entry/gregg-pop...

```

```

45745 https://www.huffingtonpost.com/entry/rafal-lip...
45746 https://www.huffingtonpost.com/entry/san-jose-...
45747 https://www.huffingtonpost.com/entry/dab-kid-s...
45748 https://www.huffingtonpost.com/entry/the-first...

                                short_description \
0      Resting is part of training. I've confirmed wh...
1      Think of talking to yourself as a tool to coac...
2      The clock is ticking for the United States to ...
3      If you want to be busy, keep trying to be perf...
4      First, the bad news: Soda bread, corned beef a...
...
45744 Monday's events beg the question -- would a bl...
45745      Plenty of air, plenty to do
45746 Many fans were pissed after seeing the minor l...
45747      Never change, young man. Never change.
45748      The gymnast is in a league of her own.

                                keywords
0                                running-lessons
1                                talking-to-yourself-crazy
2                                crenezumab-alzheimers-disease-drug
3                                meaningful-life
4                                green-superfoods
...
45744 gregg-popovich-george-hill-hair-blond
45745 rafal-lipinski-nails-sick-complicated-dunk
45746 san-jose-giants-japanese-heritage-night
45747 dab-kid-san-antonio-spurs
45748 the-first-simone-biles

```

```
[45749 rows x 5 columns]
```

```
[39]: # dropping these two columns not required for classification
df = df.drop(['keywords', 'links'], axis=1)
```

```
[40]: # checking nulls
df.isnull().sum()
```

```
[40]: category          0
      headline          0
      short_description  6
      dtype: int64
```

```
[41]: df[df['short_description'].isnull()]
```

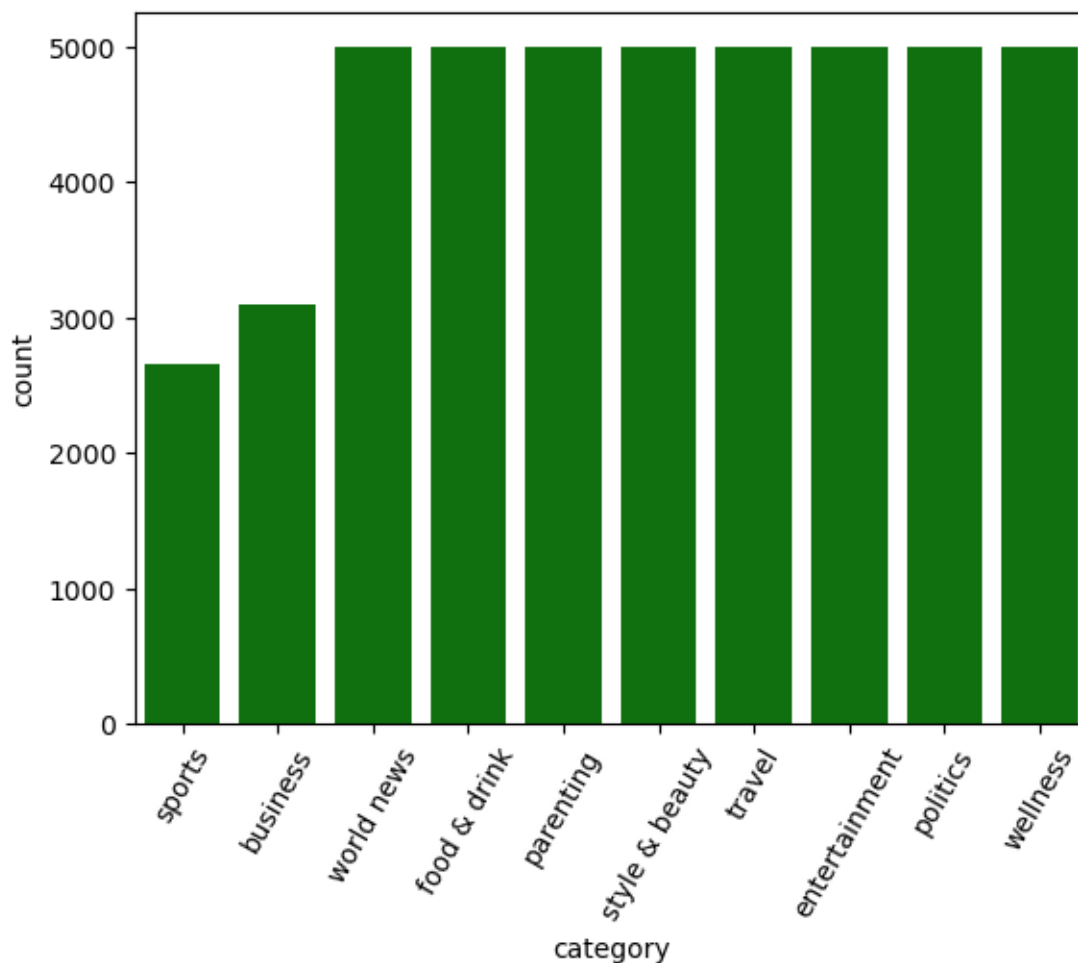
```
[41]:
```

	category	headline \
135	WELLNESS	The Health Gains -- And Strains -- Of Ultrarun...
15112	TRAVEL	National Parks Showdown: The United States vs...
17282	TRAVEL	4 Cruise Ship Ports That Suck
17642	TRAVEL	Happy Memories of A Trip to Southwest France: ...
17754	TRAVEL	A Room with a Show: The World's Top Hotel Ente...
22517	STYLE & BEAUTY	North West Makes Her Debut And More Instagrams...

	short_description
135	NaN
15112	NaN
17282	NaN
17642	NaN
17754	NaN
22517	NaN

```
[74]: import matplotlib.pyplot as plt
import seaborn as sns

sns.countplot(x=df['category'], color = 'green', order=df['category'].
    ↪value_counts().index[::-1]) # ascending
plt.xticks(rotation=60)
plt.show()
```



To understand the dataset better, I performed EDA on the distribution of categories. The countplot shows that categories such as world news, entertainment, politics, and travel contain the highest number of samples, while sports and business have comparatively fewer entries. This helps identify slight class imbalance and justifies the use of `class_weight='balanced'` in model training.

The dataset is well-balanced across all categories, with each category containing approximately the same number of news articles. This ensures fair training for the classification model.

```
[42]: # filling description columns with ''
df['short_description'] = df['short_description'].fillna('')
```

```
[43]: # creating a function to transform into lower case
```

```
def low(df):
    for i in df.columns:
        if df[i].dtypes == 'object':
            df[i] = df[i].str.lower()
```

```
return df
```

```
[44]: low(df)
```

```
[44]:      category      headline \
0      wellness      143 miles in 35 days: lessons learned
1      wellness      talking to yourself: crazy or crazy helpful?
2      wellness      crenezumab: trial will gauge whether alzheimer...
3      wellness      oh, what a difference she made
4      wellness      green superfoods
...
45744      sports      gregg popovich wishes he 'had the guts' to dye...
45745      sports      rafal lipinski nails sick, complicated dunk
45746      sports      this baseball team learned there's a wrong way...
45747      sports      some young spurs fan dabbed 38 times in a sing...
45748      sports      simone biles isn't the next anyone, she's 'the...

      short_description
0      resting is part of training. i've confirmed wh...
1      think of talking to yourself as a tool to coac...
2      the clock is ticking for the united states to ...
3      if you want to be busy, keep trying to be perf...
4      first, the bad news: soda bread, corned beef a...
...
45744      monday's events beg the question -- would a bl...
45745      plenty of air, plenty to do
45746      many fans were pissed after seeing the minor l...
45747      never change, young man. never change.
45748      the gymnast is in a league of her own.

[45749 rows x 3 columns]
```

```
[45]: # combining the text columns
df['combined_text'] = df['headline'] + df['short_description']
df['combined_text'].head()
```

```
[45]: 0      143 miles in 35 days: lessons learnedresting i...
1      talking to yourself: crazy or crazy helpful?th...
2      crenezumab: trial will gauge whether alzheimer...
3      oh, what a difference she madeif you want to b...
4      green superfoodsfirst, the bad news: soda brea...
Name: combined_text, dtype: object
```

```
[46]: # 2. Remove punctuation and special characters.
import re #(regex)
```

```
df['combined_text'] = df['combined_text'].apply(lambda x: re.
↳sub(r'[~A-Za-z\s]', '', x)) # ^A-Za-z, it means remove everthing except A-Z
↳and a-z
# \ is an excape character which tells python to treat the character beside it
↳as a special character. So \s means, single space.
#So im telling python remove anything except alphabets from A to Z and single
↳space
print(df.head())
```

```
category                                headline \
0 wellness                143 miles in 35 days: lessons learned
1 wellness            talking to yourself: crazy or crazy helpful?
2 wellness  crenezumab: trial will gauge whether alzheimer...
3 wellness                                oh, what a difference she made
4 wellness                                green superfoods
```

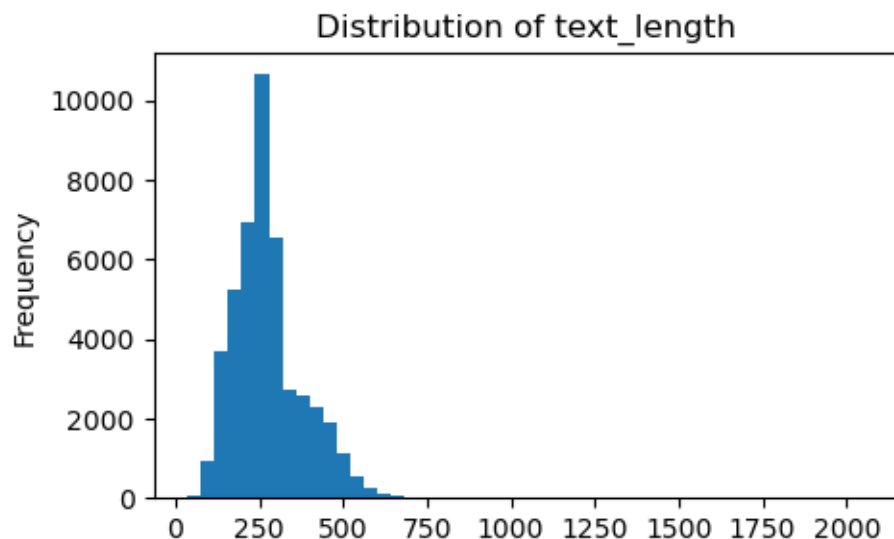
```
short_description \
0 resting is part of training. i've confirmed wh...
1 think of talking to yourself as a tool to coac...
2 the clock is ticking for the united states to ...
3 if you want to be busy, keep trying to be perf...
4 first, the bad news: soda bread, corned beef a...
```

```
combined_text
0  miles in  days lessons learnedresting is part...
1  talking to yourself crazy or crazy helpfulthin...
2  crenezumab trial will gauge whether alzheimers...
3  oh what a difference she madeif you want to be...
4  green superfoodsfirst the bad news soda bread ...
```

```
[79]: df['text_length'] = df['combined_text'].apply(lambda x: len(str(x.split())))
```

```
[80]: df['text_length'].plot(kind='hist', bins=50, figsize=(5,3), title='Distribution
↳of text_length')
```

```
[80]: <Axes: title={'center': 'Distribution of text_length'}, ylabel='Frequency'>
```



The distribution is heavily concentrated between 200–350 words, indicating that the majority of news articles fall within this length range.

```
[47]: # first I am tokenizing
from nltk.tokenize import word_tokenize
df['tokens'] = df['combined_text'].apply(lambda x: word_tokenize(x.lower()))
print(df)
```

	category	headline \
0	wellness	143 miles in 35 days: lessons learned
1	wellness	talking to yourself: crazy or crazy helpful?
2	wellness	crenezumab: trial will gauge whether alzheimer...
3	wellness	oh, what a difference she made
4	wellness	green superfoods
...	...	...
45744	sports	gregg popovich wishes he 'had the guts' to dye...
45745	sports	rafal lipinski nails sick, complicated dunk
45746	sports	this baseball team learned there's a wrong way...
45747	sports	some young spurs fan dabbed 38 times in a sing...
45748	sports	simone biles isn't the next anyone, she's 'the...
		short_description \
0		resting is part of training. i've confirmed wh...
1		think of talking to yourself as a tool to coac...
2		the clock is ticking for the united states to ...
3		if you want to be busy, keep trying to be perf...
4		first, the bad news: soda bread, corned beef a...
...		...
45744		monday's events beg the question -- would a bl...


```

45745          plenty of air, plenty to do
45746 many fans were pissed after seeing the minor l...
45747          never change, young man. never change.
45748          the gymnast is in a league of her own.

```

```

                                combined_text \
0      miles in  days lessons learnedresting is part...
1      talking to yourself crazy or crazy helpfulthin...
2      crenezumab trial will gauge whether alzheimers...
3      oh what a difference she madeif you want to be...
4      green superfoodsfirst the bad news soda bread ...
...
45744 gregg popovich wishes he had the guts to dye h...
45745 rafal lipinski nails sick complicated dunkplen...
45746 this baseball team learned theres a wrong way ...
45747 some young spurs fan dabbed  times in a single...
45748 simone biles isnt the next anyone shes the fir...

```

```

                                tokens
0      [miles, in, days, lessons, learnedresting, is,...
1      [talking, to, yourself, crazy, or, crazy, help...
2      [crenezumab, trial, will, gauge, whether, alzh...
3      [oh, what, a, difference, she, madeif, you, wa...
4      [green, superfoodsfirst, the, bad, news, soda,...
...
45744 [gregg, popovich, wishes, he, had, the, guts, ...
45745 [rafal, lipinski, nails, sick, complicated, du...
45746 [this, baseball, team, learned, theres, a, wro...
45747 [some, young, spurs, fan, dabbed, times, in, a...
45748 [simone, biles, isnt, the, next, anyone, shes,...

```

[45749 rows x 5 columns]

```

[48]: # now I will remove the stop words then I will lemmitize
from nltk.corpus import stopwords
stop_words = set(stopwords.words('English'))
df['after_stop'] = df['tokens'].apply(lambda x: [i for i in x if i not in_
↳stop_words])

print('Before stopwatch: ', df['tokens'].head())
print('After stopwatch: ', df['after_stop'].head())

```

```

Before stopwatch: 0      [miles, in, days, lessons, learnedresting, is,...
1      [talking, to, yourself, crazy, or, crazy, help...
2      [crenezumab, trial, will, gauge, whether, alzh...
3      [oh, what, a, difference, she, madeif, you, wa...
4      [green, superfoodsfirst, the, bad, news, soda,...
Name: tokens, dtype: object

```

```

After stopwatch: 0      [miles, days, lessons, learnedresting, part, t...
1      [talking, crazy, crazy, helpfulthink, talking,...
2      [crenezumab, trial, gauge, whether, alzheimers...
3      [oh, difference, madeif, want, busy, keep, try...
4      [green, superfoodsfirst, bad, news, soda, brea...
Name: after_stop, dtype: object

```

```

[49]: # applying lemmatization with POS tags
from nltk import pos_tag, word_tokenize
from nltk.corpus import wordnet
from nltk.stem import WordNetLemmatizer

df['pos_tags'] = df['after_stop'].apply(lambda x: pos_tag(x))

def get_wordnet_pos(tag):
    if tag.startswith('J'): return wordnet.ADJ
    elif tag.startswith('V'): return wordnet.VERB
    elif tag.startswith('N'): return wordnet.NOUN
    elif tag.startswith('R'): return wordnet.ADV
    else: return wordnet.NOUN

lemm = WordNetLemmatizer()
df['lemm_with_pos'] = df['pos_tags'].apply(lambda x: [lemm.lemmatize(word,
    get_wordnet_pos(pos)) for (word, pos) in x])
print(df[['after_stop', 'lemm_with_pos']].head())

```

```

                                after_stop \
0  [miles, days, lessons, learnedresting, part, t...
1  [talking, crazy, crazy, helpfulthink, talking,...
2  [crenezumab, trial, gauge, whether, alzheimers...
3  [oh, difference, madeif, want, busy, keep, try...
4  [green, superfoodsfirst, bad, news, soda, brea...

                                lemm_with_pos
0  [mile, day, lesson, learnedresting, part, trai...
1  [talk, crazy, crazy, helpfulthink, talk, tool,...
2  [crenezumab, trial, gauge, whether, alzheimers...
3  [oh, difference, madeif, want, busy, keep, try...
4  [green, superfoodsfirst, bad, news, soda, brea...

```

```

[50]: # I'll apply word embedding now
# training the model
from gensim.models import Word2Vec
import numpy as np

model = Word2Vec(sentences=df['lemm_with_pos'].tolist(), vector_size=100,
    window=3, min_count=1, sg=1)

```

```

#Here vector_size=100 means 100 vectors
#window= 3, it will look for 3 words before and after the target word when
↳training the model.
'''sk=1 The Skip-Gram model takes every single word in the training text and
↳treats it as a target word, one by one.
For each target word, it tries to predict the nearby words (called context
↳words) within the defined window size.'''

#min_count=1 It tell the model to ignore all words that appear less than this
↳number of times in the training text

# tolist will convert a panda series, 1D array into a python list

```

```

[50]: 'sk=1 The Skip-Gram model takes every single word in the training text and
treats it as a target word, one by one.\nFor each target word, it tries to
predict the nearby words (called context words) within the defined window size.'

```

```

[51]: # Computing the sentence embedding
embeddings = []

for i in df['lemm_with_pos']:
    word_embeddings = [model.wv[j] for j in i if j in model.wv]

    if word_embeddings:
        sentence_embedding = np.mean(word_embeddings, axis=0)
    else:
        sentence_embedding = np.zeros(model.vector_size)

    embeddings.append(sentence_embedding)

```

```

[52]: #convert into array
import numpy as np

embedding_array = np.array(embeddings)
print(embedding_array)

```

```

[[-2.18513533e-01  2.86657870e-01  1.04115658e-01 ... -4.81082201e-01
  9.08440650e-02 -1.46441430e-01]
 [-1.16074733e-01  2.79760957e-01  1.12803544e-04 ... -3.73269528e-01
  3.62136289e-02 -1.72091514e-01]
 [-1.26194566e-01  3.17227542e-01  3.86212766e-02 ... -4.38105762e-01
  5.85949495e-02 -1.83596164e-01]
 ...
 [-1.52107120e-01  2.74074078e-01  6.12128899e-02 ... -4.27465022e-01
  1.84930295e-01 -1.02470487e-01]
 [-1.50187343e-01  3.03167522e-01  5.25261983e-02 ... -4.66479570e-01
  1.81133032e-01 -2.38500819e-01]

```

```
[-7.21052215e-02  2.58209050e-01  6.41121045e-02 ... -3.50629091e-01
 1.03427224e-01 -1.08648010e-01]]
```

```
[53]: df['lemm_with_pos']
```

```
[53]: 0      [mile, day, lesson, learnedresting, part, trai...
1      [talk, crazy, crazy, helpfulthink, talk, tool,...
2      [crenezumab, trial, gauge, whether, alzheimers...
3      [oh, difference, madeif, want, busy, keep, try...
4      [green, superfoodsfirst, bad, news, soda, brea...

...
45744   [gregg, popovich, wish, gut, dye, hair, blondm...
45745   [rafal, lipinski, nail, sick, complicate, dunk...
45746   [baseball, team, learn, there, wrong, way, cel...
45747   [young, spur, fan, dab, time, single, playoff,...
45748   [simone, bile, isnt, next, anyone, shes, first...
Name: lemm_with_pos, Length: 45749, dtype: object
```

```
[54]: # Joining the tokens back into a sentence for TF-IDF.
# TF-IDF works on complete text strings, not on lists of individual tokens.
# Word2Vec uses a list of tokens, but TF-IDF requires the tokens to be joined
↳back into one sentence.ist
df['lemm'] = df['lemm_with_pos'].apply(lambda x: ' '.join(x) if isinstance(x,
↳(tuple,list)) else ('' if x is None else str(x)))
```

```
[55]: df['lemm']
```

```
[55]: 0      mile day lesson learnedresting part train ive ...
1      talk crazy crazy helpfulthink talk tool coach ...
2      crenezumab trial gauge whether alzheimers drug...
3      oh difference madeif want busy keep try perfec...
4      green superfoodsfirst bad news soda bread corn...

...
45744   gregg popovich wish gut dye hair blondmondays ...
45745   rafal lipinski nail sick complicate dunkplenty...
45746   baseball team learn there wrong way celebrate ...
45747   young spur fan dab time single playoff gamenev...
45748   simone bile isnt next anyone shes first simone...
Name: lemm, Length: 45749, dtype: object
```

```
[56]: ## Apply Tf-IDF
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['lemm'])

tfidf_features = vectorizer.get_feature_names_out()
```

```
print('shape:', tfidf_matrix.shape)
```

shape: (45749, 83264)

```
[57]: # applying label encoding on label columns to convert it into 1 and 0
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
y_encoded = label.fit_transform(df['category'])
```

```
[58]: print(label.classes_, label.transform(label.classes_))
```

['business' 'entertainment' 'food & drink' 'parenting' 'politics' 'sports'
 'style & beauty' 'travel' 'wellness' 'world news'] [0 1 2 3 4 5 6 7 8 9]

```
[59]: # sepearting independant and dependant variables
x = tfidf_matrix
y = y_encoded
```

```
[60]: # I will split the data into 80% training 20 % testing

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
↳random_state=42, stratify=y)
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

(36599, 83264)

(9150, 83264)

(36599,)

(9150,)

```
[61]: from sklearn.svm import LinearSVC # used linear SVC than SVC(kernel='rbf')
↳because it cannot handle high dimensions data, it becomes extremely slow or
↳impossible to train.
# LinearSVC is designed for high-dimensional sparse data (like text)
from sklearn.metrics import accuracy_score, classification_report,
↳confusion_matrix

sv = LinearSVC(class_weight='balanced', random_state=42)
sv.fit(x_train, y_train)
```

```
[61]: LinearSVC(class_weight='balanced', random_state=42)
```

```
[62]: # lets check can it predict well or not
y_predict = sv.predict(x_test)
```

```
[63]: #checking scores
print(sv.score(x_train,y_train))
print(sv.score(x_test,y_test))
```

```
0.9909013907483811
0.7853551912568306
```

```
[64]: # evaluating
from sklearn.metrics import accuracy_score, confusion_matrix, \
    classification_report

accuracy_score = accuracy_score(y_test, y_predict)
confusion_matrix = confusion_matrix(y_test, y_predict)
classification_report = classification_report(y_test, y_predict)

print(f'accuracy_score is {accuracy_score}')
print(f'confusion_matrix is {confusion_matrix}')
print(f'classification_report is {classification_report}')
```

```
accuracy_score is 0.7853551912568306
confusion_matrix is [[403  15  21  19  44   7  10  22  55  22]
 [ 13 789  16  35  28  26  38  18  23  14]
 [ 10  14 875  17   3   8  11  31  28   3]
 [ 26  39  15 777  19  11  21  24  64   4]
 [ 53  27   4  22 768  13   4  16  22  71]
 [  7  36   5  10  19 411   8   6  13  17]
 [  8  45  16  23   9   4 830  31  25   9]
 [ 19  18  53  24  14  20  22 767  28  35]
 [ 41  12  56  68  27  16   8  22 738  12]
 [ 30  12   8  12  41  10   6  35  18 828]]
classification_report is
```

		precision	recall	f1-score	support
0	0.66	0.65	0.66	618	
1	0.78	0.79	0.79	1000	
2	0.82	0.88	0.85	1000	
3	0.77	0.78	0.77	1000	
4	0.79	0.77	0.78	1000	
5	0.78	0.77	0.78	532	
6	0.87	0.83	0.85	1000	
7	0.79	0.77	0.78	1000	
8	0.73	0.74	0.73	1000	
9	0.82	0.83	0.82	1000	
accuracy		0.79		9150	

macro avg	0.78	0.78	0.78	9150
weighted avg	0.79	0.79	0.79	9150

```
[86]: import numpy as np

# Get feature names
feature_names = vectorizer.get_feature_names_out()

# Number of top words
TOP_N = 3

for i, class_name in enumerate(label.classes_): # enumerate(label.classes_)
    ↪ gives both the index and the class name.
    print('Category:', class_name)
    class_coef = sv.coef_[i]
    top_indices = class_coef.argsort()[-TOP_N:] [::-1]
    top_words = feature_names[top_indices]
    print('Top keywords:', list(top_words))

# enumerate see below what it does
# i = 0, class_name = 'business'
# i = 1, class_name = 'sports'
```

```
Category: business
Top keywords: ['wall', 'krugman', 'business']
Category: entertainment
Top keywords: ['album', 'actor', 'trailer']
Category: food & drink
Top keywords: ['recipe', 'cocktail', 'wine']
Category: parenting
Top keywords: ['parent', 'kid', 'breastfeed']
Category: politics
Top keywords: ['gop', 'senate', 'republican']
Category: sports
Top keywords: ['nfl', 'nba', 'lebron']
Category: style & beauty
Top keywords: ['fashion', 'style', 'dress']
Category: travel
Top keywords: ['travel', 'traveler', 'hotel']
Category: wellness
Top keywords: ['workout', 'fearless', 'spiritual']
Category: world news
Top keywords: ['saudi', 'refugee', 'korea']
```

To understand which features drive the model’s decisions, I extracted the top keywords for each news category using the coefficient weights of the LinearSVC classifier. The model correctly identifies category-specific terms such as “nfl”, “nba”, “lebron” for sports, “recipe”, “cocktail”, “wine” for

food & drink, and “gop”, “senate”, “republican” for politics. These keywords represent the most influential terms that guide the classifier’s predictions. This satisfies the success requirement of deriving meaningful insights about which words contribute most to each category

```
[65]: # trying with random forest
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report, \
    confusion_matrix

# base model
rf = RandomForestClassifier(random_state=42, class_weight='balanced')

# Parameter grid
rf_param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None],
    'min_samples_split': [5],
    'min_samples_leaf': [2],
    'max_features': ['sqrt']
}
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

# GridSearchCV
rf_grid = GridSearchCV(estimator=rf, param_grid=rf_param_grid, \
    scoring='f1_macro', cv=cv, n_jobs=-1, verbose=1)

rf_grid.fit(x_train, y_train)

# Get the best model
print('RF best params', rf_grid.best_params_)
best_rf = rf_grid.best_estimator_
```

Fitting 3 folds for each of 2 candidates, totalling 6 fits

RF best params {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 200}

```
[66]: # lets check can it predict well or not

y_predict1 = best_rf.predict(x_test)
```

```
[67]: #checking scores
print(best_rf.score(x_train,y_train))
print(best_rf.score(x_test,y_test))
```

0.8080548648870187
0.7174863387978142


```
[68]: # evaluating
from sklearn.metrics import accuracy_score, confusion_matrix, \
    classification_report

accuracy_score = accuracy_score(y_test, y_predict1)
confusion_matrix = confusion_matrix(y_test, y_predict1)
classification_report = classification_report(y_test, y_predict1)

print(f'accuracy_score is {accuracy_score}')
print(f'confusion_matrix is {confusion_matrix}')
print(f'classification_report is {classification_report}')
```

```
accuracy_score is 0.7174863387978142
confusion_matrix is [[366  19  20  18  46  19  12  28  64  26]
 [ 18 654  29  70  37  48  51  33  36  24]
 [ 14  24 853  21   5  13  18  25  21   6]
 [ 13  26  22 773  12  13  39  20  72  10]
 [ 57  33  10  28 665  17  10  27  52 101]
 [ 10  30   7  18  17 385  12  14  17  22]
 [ 17  56  30  31  15  17 765  22  40   7]
 [ 24  24  73  29  16  20  41 710  30  33]
 [ 30  30  81  67  17  26  22  34 674  19]
 [ 36  19  13  21  71  23  11  50  36 720]]
classification_report is
```

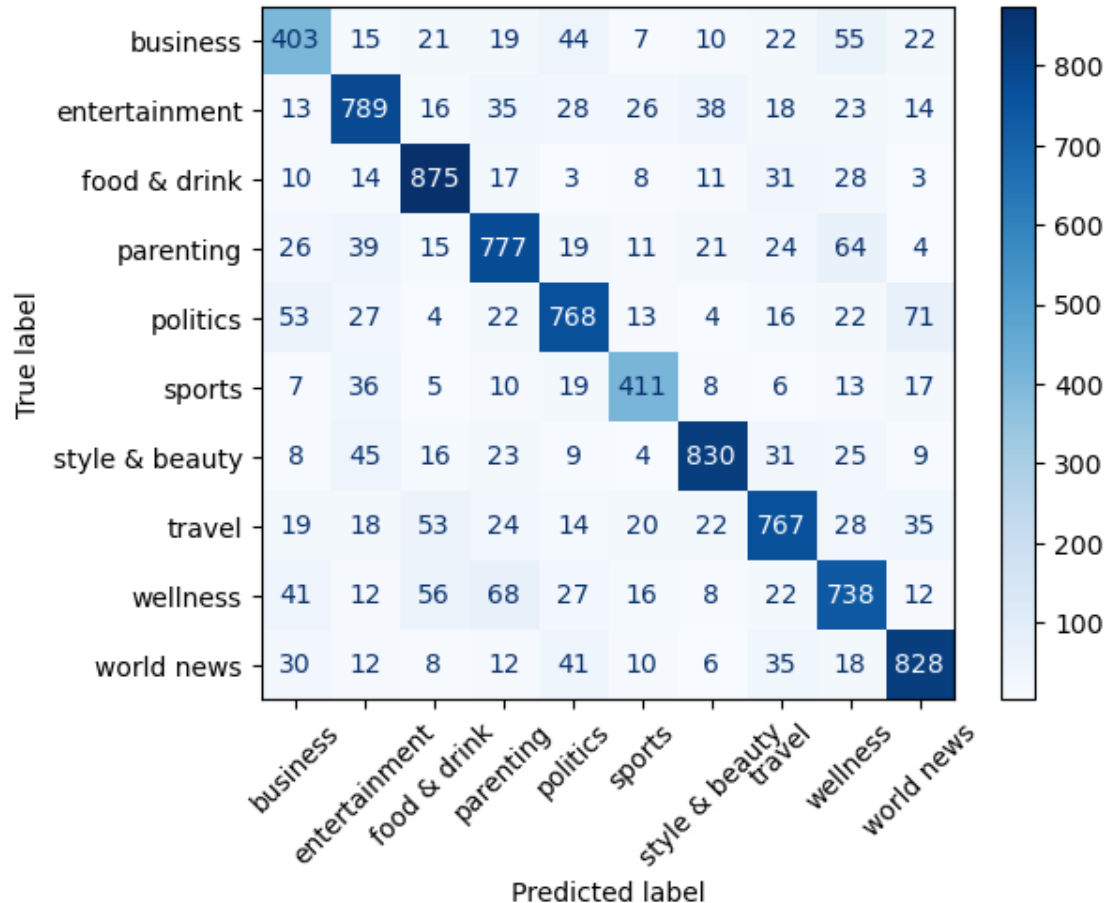
		precision	recall	f1-score	support
0	0.63	0.59	0.61	618	
1	0.71	0.65	0.68	1000	
2	0.75	0.85	0.80	1000	
3	0.72	0.77	0.74	1000	
4	0.74	0.67	0.70	1000	
5	0.66	0.72	0.69	532	
6	0.78	0.77	0.77	1000	
7	0.74	0.71	0.72	1000	
8	0.65	0.67	0.66	1000	
9	0.74	0.72	0.73	1000	
accuracy		0.72		9150	
macro avg	0.71	0.71	0.71	9150	
weighted avg	0.72	0.72	0.72	9150	

```
[78]: # displaying the confusion matrix of svm model the result was best among all of \
    my other models
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
import matplotlib.pyplot as plt
cm = confusion_matrix(y_test, y_predict) # used logistic model
```

```

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels= label.
    classes_)
disp.plot(cmap='Blues', xticks_rotation=45, values_format='d')
plt.show()

```



The confusion matrix above shows how well the LinearSVC model classified each news category. The diagonal blocks contain the highest values, meaning most predictions match the true labels. Categories such as entertainment, food & drink, style & beauty, travel, and world news show very high correct prediction counts. A few categories, like business and sports, have more misclassifications, but overall the model performs strongly across all classes. This visualization confirms that the SVM model is effective at multi-class news classification.

```

[81]: # predict
import re
import pandas as pd
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords, wordnet
from nltk import pos_tag

```

```

from nltk.stem import WordNetLemmatizer

# Step 1:
# pos tags
def _wn_pos(tag):
    if tag.startswith('J'): return wordnet.ADJ
    if tag.startswith('V'): return wordnet.VERB
    if tag.startswith('N'): return wordnet.NOUN
    if tag.startswith('R'): return wordnet.ADV
    return wordnet.NOUN

#lower case and removing speacial charcaters
def preprocess_text(text):
    if text is None:
        return ''
    text = text.lower()
    text = re.sub(r'[^A-Za-z\s]', ' ', text).strip()

    # removing stopwords
    sw = set(stopwords.words('english'))
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in sw]

    # applying lemmitization
    lemm = WordNetLemmatizer()
    tagged = pos_tag(tokens)
    lemmas = [lemm.lemmatize(w, _wn_pos(t)) for w, t in tagged]

    return ' '.join(lemmas)

# Step 2: TF-IDF + Predict function
def predict_category_TFIDF(text_list, vectorizer, sv, label):
    """
    text_list: list of strings (new headlines or combined_text)
    vectorizer: fitted TfidfVectorizer from training
    model: trained SVM model (LinearSVC or SVC)
    label_encoder: fitted LabelEncoder used during training
    """

    # Preprocess each text exactly like training
    cleaned = [preprocess_text(t) for t in text_list]

    # TF-IDF transform (important: NO FITTING!)
    x = vectorizer.transform(cleaned)

```

```

# Predict numeric labels
preds = sv.predict(x)

# Convert to actual category names
final_labels = label.inverse_transform(preds)

return final_labels

```

```

[82]: new_data = [
        "Government bans new crypto policies",
        "Lionel Messi wins another award",
        "Healthy breakfast foods you must try"
    ]

predicted_categories = predict_category_TFIDF(new_data, vectorizer, sv, label)
print(predicted_categories)

```

```
['world news' 'sports' 'wellness']
```

To test the model's real-world usefulness, a few unseen news headlines were passed through the same preprocessing and TF-IDF transformation pipeline used during training. The trained LinearSVC model then predicted the most likely category for each article. The model correctly classified the examples as 'world news', 'sports', and 'wellness', showing that it can generalize well to new, unseen text. This demonstrates that the classifier is able to understand the context of different news topics and assign appropriate categories based on the content of the article.

Summary of the Part B

In Part B, I performed multi-class news article classification using a dataset of 45k+ samples. I cleaned and preprocessed the text by removing punctuation, converting to lowercase, tokenizing, removing stopwords, and performing POS-based lemmatization. I applied TF-IDF vectorization to transform text into numerical features (83k dimensions) and label-encoded the categories. I trained LinearSVC and a tuned Random Forest model. LinearSVC achieved the highest performance with 78.5% accuracy on test data. Evaluation included accuracy, confusion matrix, and classification report. I also built a custom prediction function to classify new unseen news headlines, demonstrating the model's practical applicability.